Project - High Level Design

On

Non-Profit Reactive Triage Agent

Course Name: **Agentic AI**

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	AAYUSH BANWADIKAR	EN22ME304002
2	DIVYANSHU MEHTA	EN22EL301024
3	ADITYA KUMAR SAHU	EN22ME304005
4	VINAY PATIDAR	EN22CS3011085
5	YATENDRA VERMA	EN22ME304118

Group Name: 02D10

Project Number: AAI-15

Industry Mentor Name: Mr. Ashwin Krishnan CK

University Mentor Name: Prof. Maya Yadav Baniya

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. State and Session Management

5. Interfaces

6. Non-Functional Requirements
   6.1. Security Aspects
   6.2. Performance Aspects

7. References

1. Introduction

1.1. Scope of the document

The Non-Profit Support Triage Agent focuses on automating the process of handling and prioritizing support requests received by non-profit organizations. The system is designed to analyze user queries written in natural language, classify them into predefined categories such as food, medical aid, shelter, and education, and determine their urgency level (high, medium, or low priority). Based on this analysis, the system assists in directing requests to the appropriate resources or departments.

The project utilizes AI technologies, including frameworks like LangChain and CrewAI, to enable intelligent decision-making and efficient query handling. It also supports basic knowledge retrieval from stored data sources to provide relevant responses.

The scope of this project is limited to text-based input and operates within predefined categories and rules. It is developed as a prototype system and does not include full integration with real-world non-profit databases or real-time emergency services. However, it establishes a foundation that can be extended in the future to include advanced features such as multilingual support, voice interaction, mobile applications, and real-time system integration.

1.2. Indented Audience

The Non-Profit Support Triage Agent is intended for non-profit organizations and NGOs that manage a large number of support requests. It helps them automate the classification and prioritization of queries, improving efficiency and response time.

The system is also useful for volunteers and support staff, as it reduces manual effort and assists in identifying urgent cases quickly. Additionally, it serves individuals seeking help by providing a structured way to submit their requests and ensuring they are directed to the appropriate resources.

Furthermore, administrators can use the system to monitor and manage requests effectively. The project is also relevant for developers and researchers working with AI technologies such as LangChain and CrewAI, as it demonstrates the practical application of intelligent systems in social support services.

1.3. System Overview

The Non-Profit Reactive Triage Agent is an AI-based system designed to automate the initial handling of communications received by non-profit organizations. It processes incoming messages, identifies their urgency and intent, extracts important information, and generates appropriate draft responses to improve efficiency and response time.

The system is built using a **multi-agent architecture**, where different agents perform specific tasks in a sequential workflow. The backend is developed using FastAPI and coordinated using CrewAI, while high-speed LLM processing is powered by Groq. For entity extraction, spaCy is used, and the user interface is built using React.

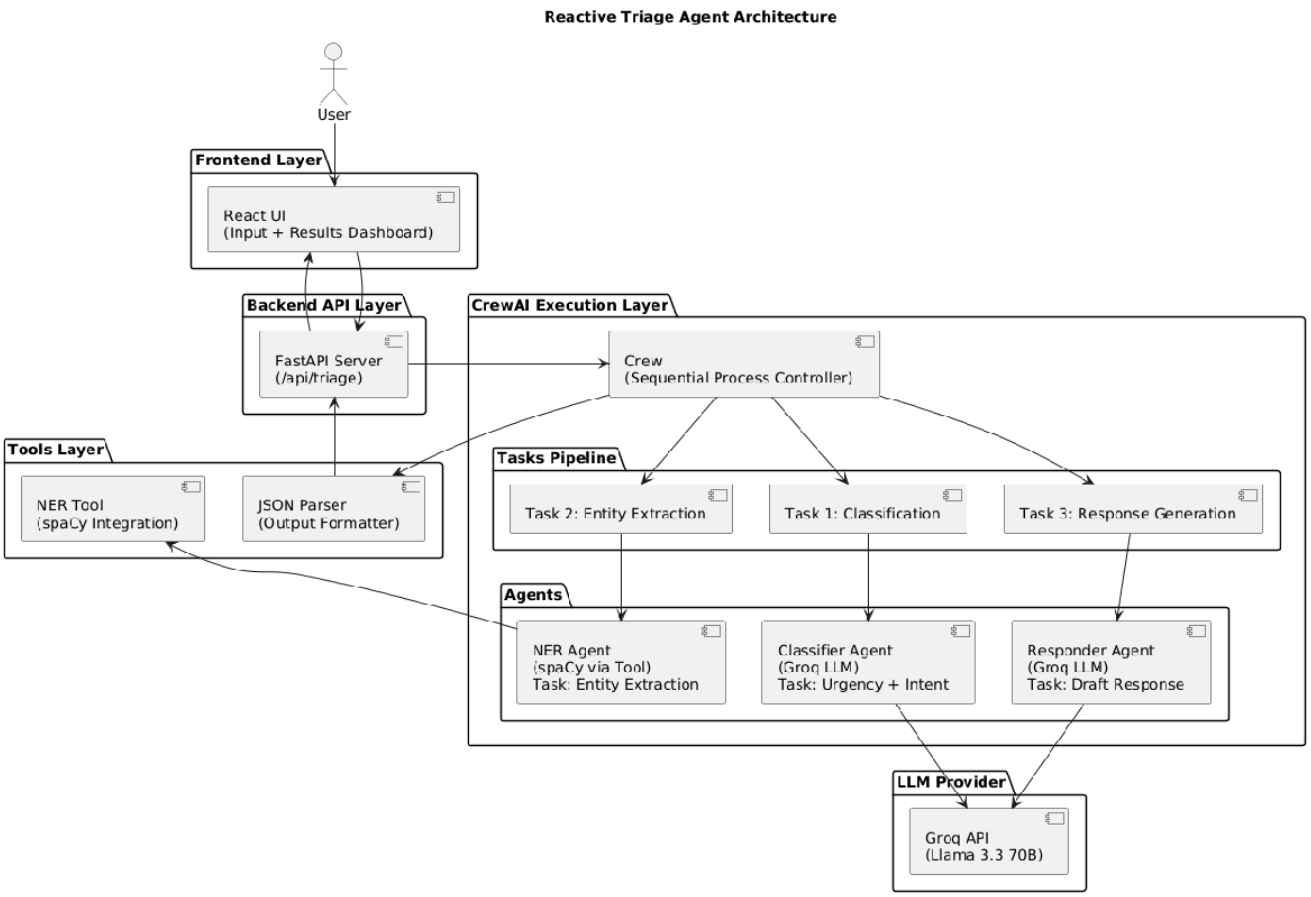
- **Automated Processing:** Automatically handles incoming messages without manual intervention
- **Multi-Agent Workflow:** Uses classifier, extractor, and responder agents for structured processing
- **Message Classification:** Determines urgency and intent (high, medium, low priority)
- **Entity Extraction (NER):** Extracts key details like names, dates, and IDs using spaCy
- **Response Generation:** Generates context-aware and meaningful replies using LLM
- **Backend Processing:** Managed using FastAPI for API handling
- **Agent Orchestration:** Controlled using CrewAI
- **Fast LLM Inference:** Powered by Groq for quick responses
- **User Interface:** Built using React for interaction

2. System Deign

2.1. Application Design

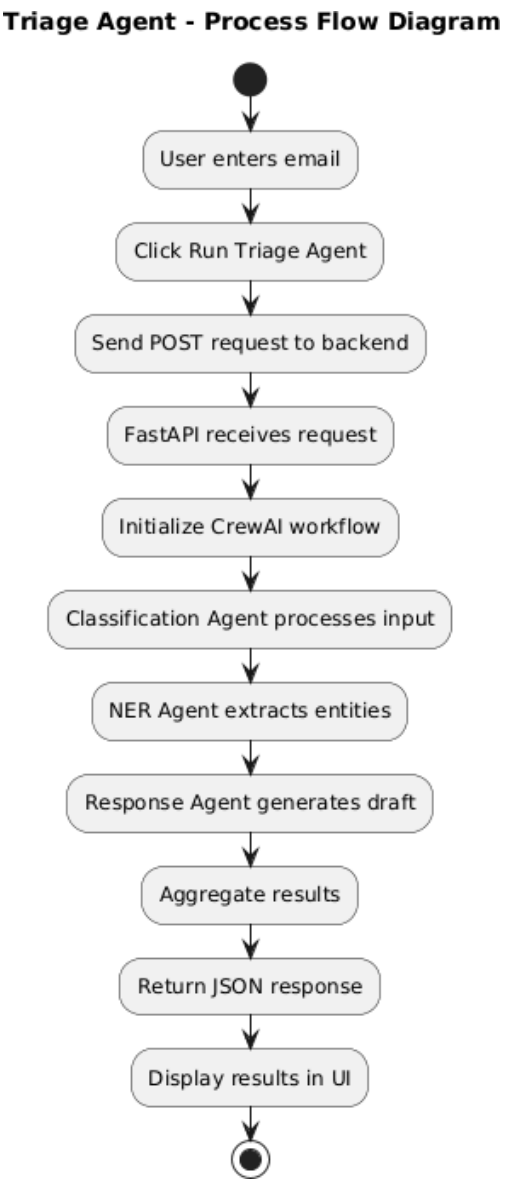
The Non-Profit Reactive Triage Agent is designed using a modular and scalable architecture that separates the system into frontend, backend, and AI-driven agent components. The design follows a structured approach where each component is responsible for a specific function, ensuring maintainability, flexibility, and efficient processing.

The application consists of a React-based frontend, a FastAPI backend, and a multi-agent system coordinated using CrewAI. The system processes user inputs through a sequential pipeline of intelligent agents to generate structured outputs.



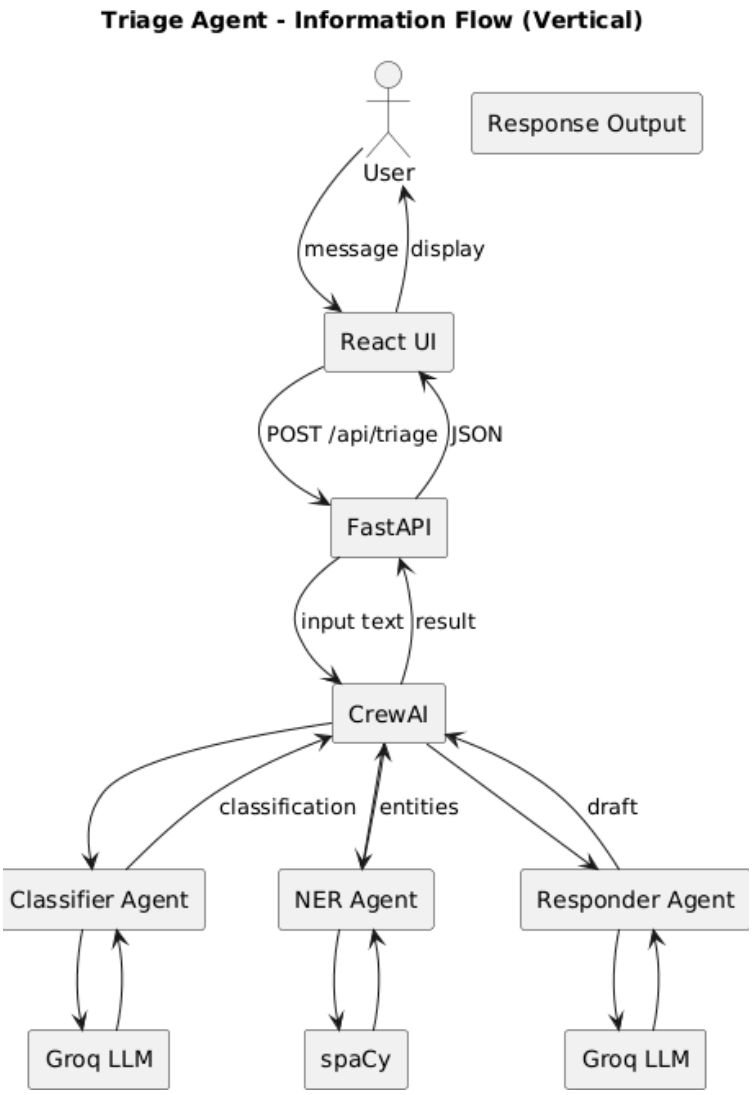
- **User Interface (Frontend):** Built using React to allow users to input messages and view results (classification, entities, responses)
- **Backend API Layer:** Developed using FastAPI to handle requests, route data, and return structured JSON responses
- **Multi-Agent System:** Implemented using CrewAI to coordinate classifier, extractor, and responder agents
- **Natural Language Processing Module:** Uses spaCy for extracting entities like names, dates, and IDs
- **LLM Integration:** Powered by Groq to perform classification and generate context-aware responses efficiently
- **Workflow Design:** Follows a sequential pipeline where input is processed through classification, entity extraction, and response generation before returning results

2.2. Process Flow

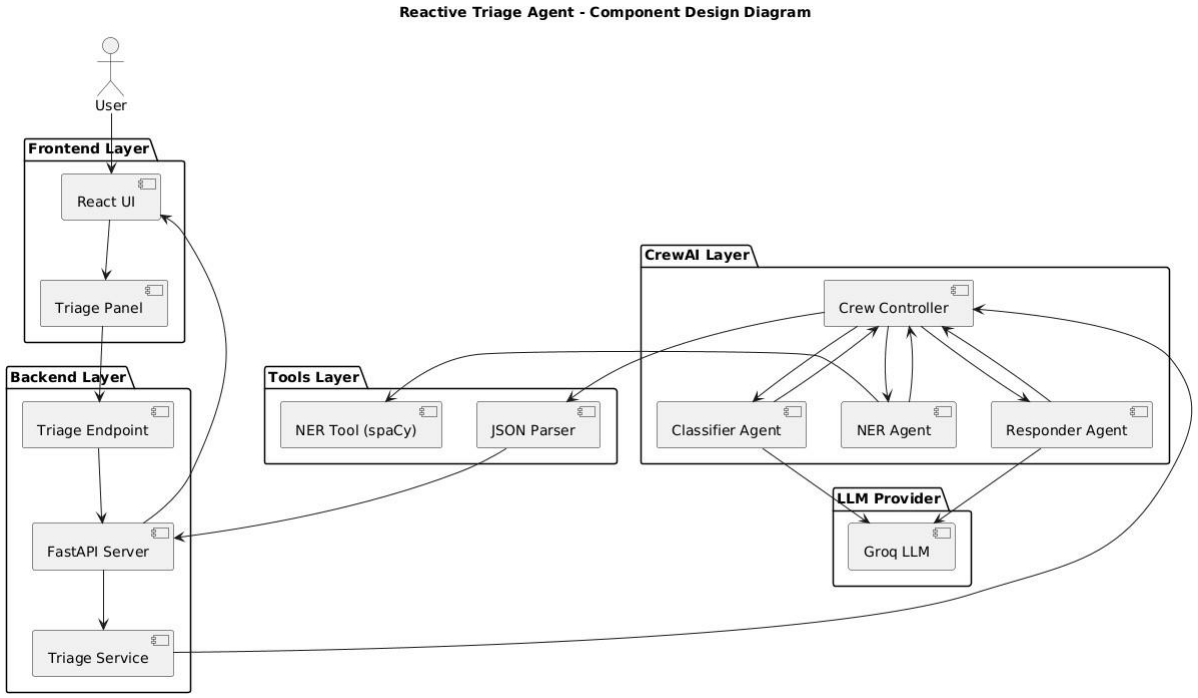


- **User Input:** User enters message in React UI and sends request to backend
- **Backend Processing:** FastAPI triggers CrewAI workflow using kickoff()
- **Agent Execution:** Classifier → Extractor → Responder process the message sequentially
- **Data Processing:** Intent, urgency, entities, and draft response are generated
- **Response Output:** Results are combined, sent back to frontend, and displayed to user

2.3. Information Flow



2.4. Components Design



2.5. Key Design Considerations

- **Modular & Agent-Based Architecture**: The system is divided into specialized agents (classifier, extractor, responder), making it flexible, reusable, and easy to maintain.
- **Sequential Workflow Design**: Tasks are executed in a proper order (classification → extraction → response), ensuring accurate and logical processing of data.
- **Scalability & Performance**: Use of FastAPI and high-speed LLM (Groq) ensures the system can handle multiple requests efficiently with low latency.
- **Reliability & Error Handling**: The system is designed to handle failures gracefully, ensuring consistent output even if some components fail.

2.6. API Catalogue

SNO	API Name	Endpoint	Method	Description	Request Body	Response
1	Triage API	/api/triage	POST	Main API to process incoming message through all agents	{ "message": "text" }	{ intent, urgency, entities, response }
2	Health Care API	/api/health	GET	Checks if backend service is running	None	{ status: "ok" }
3	Classification API	/api/classify	POST	Classifies message intent and urgency	{ "message": "text" }	{ intent: "...", urgency: "..." }
4	Extraction API	/api/extract	POST	Extracts entities using spaCy NER	{ "message": "text" }	{ entities: { name, date, org, id } }
5	Response API	/api/respond	POST	Generates draft response using LLM	{ message, intent, entities }	{ response: "generated reply" }

3. Data Design

3.1. Data Model

The data model of the Reactive Triage Agent defines how information flows, is structured, and is stored across the system. It ensures consistent communication between components and enables efficient processing of user inputs.
The system primarily works with structured JSON data, where each stage of the pipeline transforms the input into more meaningful and organized information.

3.2. Data Access Mechanism

- Data is accessed through the FastAPI backend APIs
- All data is passed in JSON format between components
- LLM (Groq) is accessed via API for classification and response
- spaCy NER is used locally to extract entities
- Vector DB (ChromaDB) is used to store and retrieve context (if used)
- System follows a stateless approach, so each request is processed independently

3.3. Data Retention Policies

- User data is processed in real-time and not stored permanently by default
- Temporary data is kept only during processing and then discarded
- Sensitive data is not retained to ensure privacy and security
- If storage is enabled, data is kept for a limited duration only
- Data handling follows basic security and privacy guidelines

3.4. Data Migration

- Data migration is minimal as the system mainly works in real-time
- If needed, data can be moved from logs or storage to databases
- Embeddings can be migrated between vector databases (e.g., ChromaDB)
- Data is transferred in JSON format for compatibility
- Can be used when upgrading systems or changing storage solutions

4. Session and State Management

4.1. Session Management

The Reactive Triage Agent follows a stateless session management approach, where each user request is processed independently without maintaining server-side session data. The frontend (React) manages user interactions and temporarily stores input and results using local state. The backend (FastAPI) handles each request separately, using a unique message or request ID to track processing. This approach improves scalability, performance, and simplifies system design, as no long-term session storage is required.

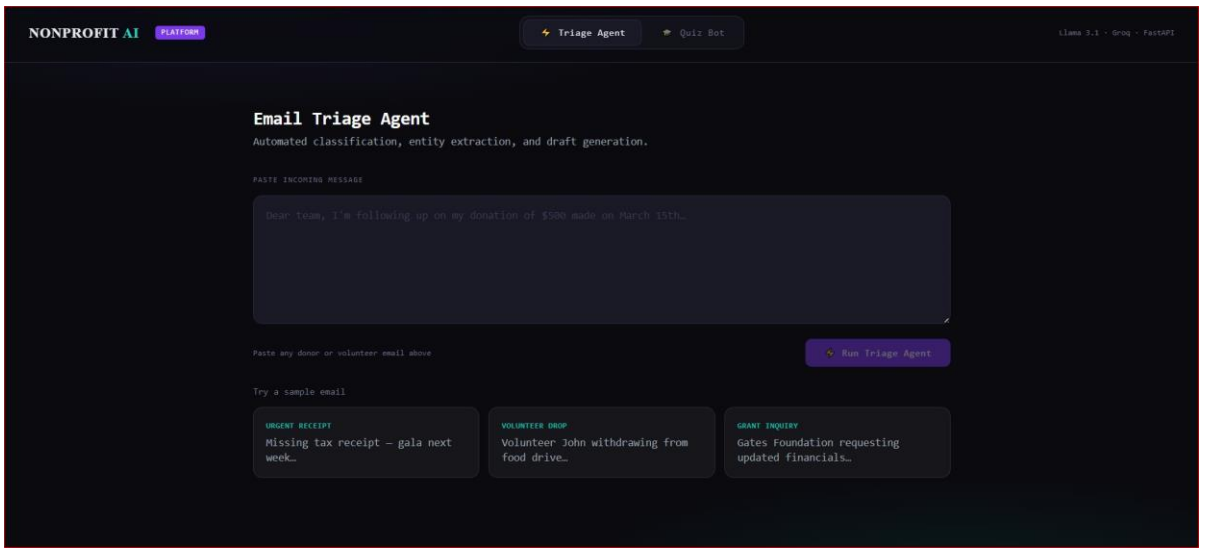
4.2. State Management

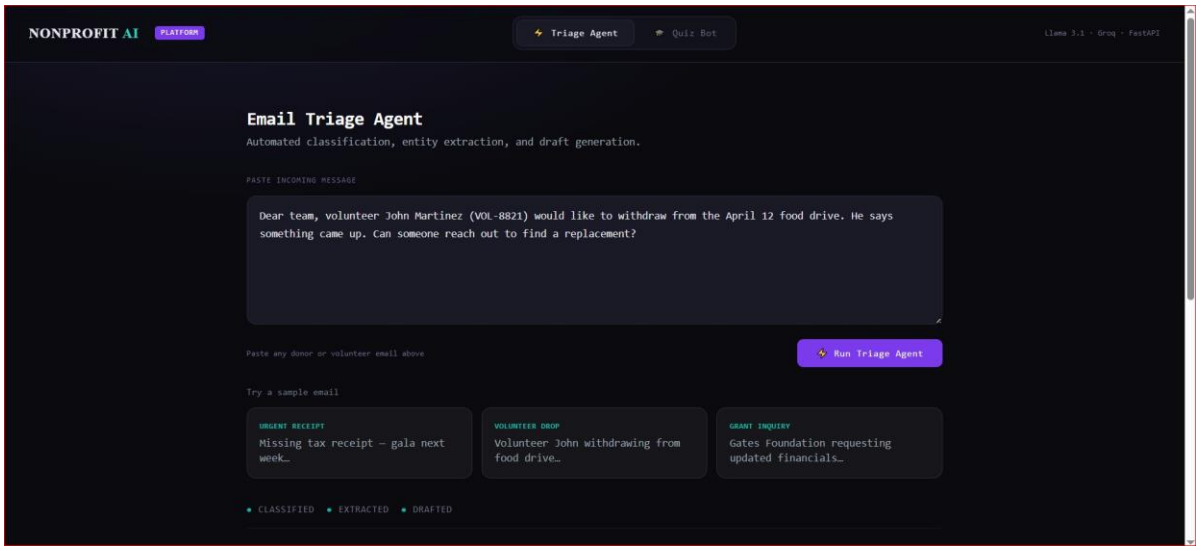
The Reactive Triage Agent uses a frontend-driven state management approach where the React application manages all user interface states. It maintains variables such as user input, loading status, processing steps, results, and error messages using component state. The backend (FastAPI) remains stateless and does not store any application state between requests. Each API call returns a complete response, which updates the frontend state accordingly. This approach ensures a responsive user experience, simplifies data handling, and improves scalability by avoiding server-side state storage.

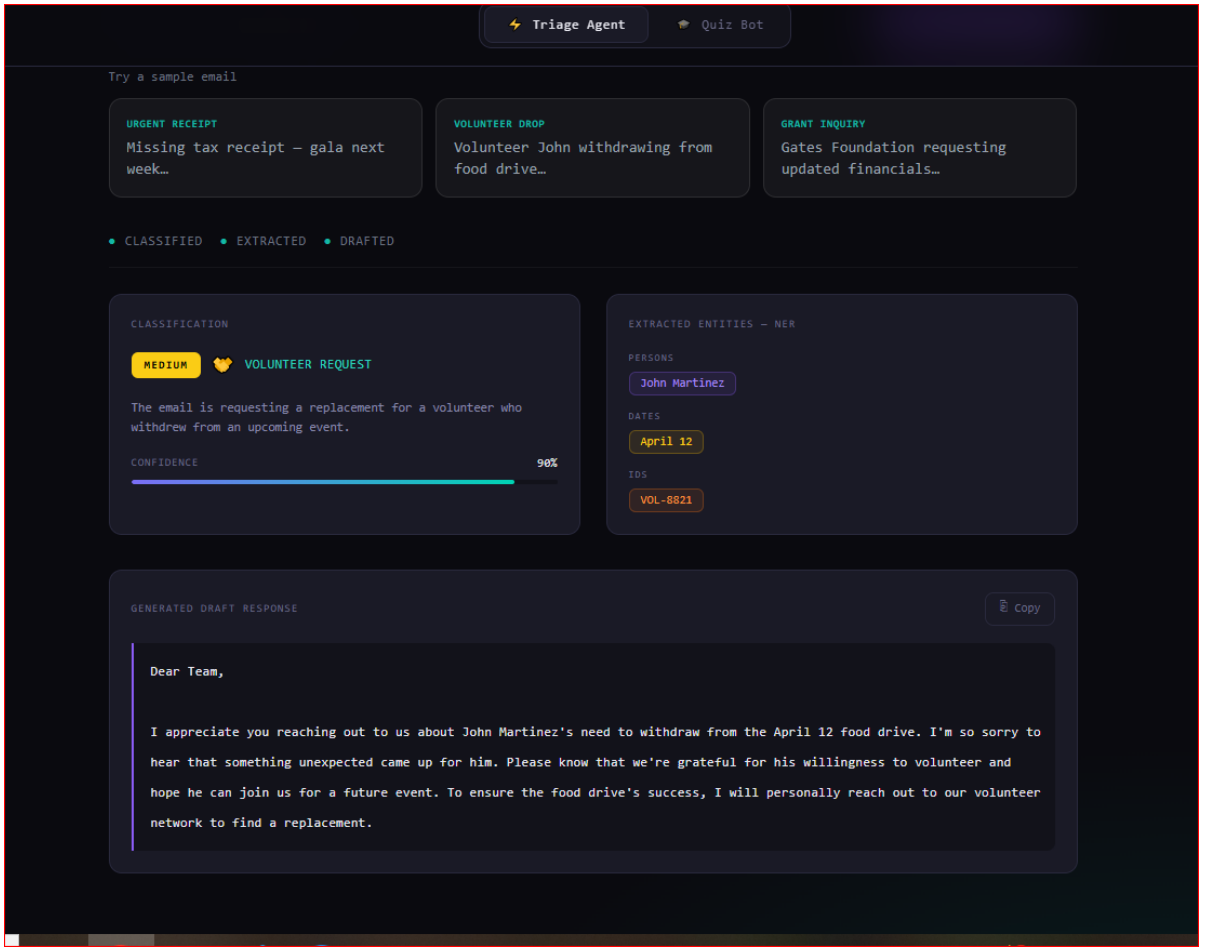
4.3. Backend-Frontend Interaction

The interaction between the frontend and backend in the Reactive Triage Agent is handled through RESTful APIs. The React frontend sends user input (message data) to the FastAPI backend using HTTP requests (e.g., POST /api/triage). The backend processes the request through the agent pipeline (classification, extraction, response generation) and returns a structured JSON response. The frontend then updates the UI to display the results such as intent, urgency, extracted entities, and generated reply. This communication is stateless, fast, and based on JSON format, ensuring smooth data exchange and scalability.

5. Interfaces




NONPROFIT AI
PLATFORM
Triage Agent
Quiz Bot
Llama 3.1 · Groq · FastAPI
Email Triage Agent
Automated classification, entity extraction, and draft generation.
PASTE INCOMING MESSAGE
Dear team, volunteer John Martinez (VOL-8821) would like to withdraw from the April 12 food drive. He says something came up. Can someone reach out to find a replacement?
Paste any donor or volunteer email above
Run Triage Agent
Try a sample email
URGENT RECEIPT
Missing tax receipt – gala next week…
VOLUNTEER DROP
Volunteer John withdrawing from food drive…
GRANT INQUIRY
Gates Foundation requesting updated financials…
CLASSIFIED
EXTRACTED
DRAFTED


Triage Agent
Quiz Bot
Try a sample email
URGENT RECEIPT
Missing tax receipt – gala next week…
VOLUNTEER DROP
Volunteer John withdrawing from food drive…
GRANT INQUIRY
Gates Foundation requesting updated financials…
CLASSIFIED
EXTRACTED
DRAFTED
CLASSIFICATION
MEDIUM
VOLUNTEER REQUEST
The email is requesting a replacement for a volunteer who withdrew from an upcoming event.
CONFIDENCE
90%
EXTRACTED ENTITIES – NER
PERSONS
John Martinez
DATES
April 12
IDS
VOL-8821
GENERATED DRAFT RESPONSE
Copy
Dear Team,
I appreciate you reaching out to us about John Martinez's need to withdraw from the April 12 food drive. I'm so sorry to hear that something unexpected came up for him. Please know that we're grateful for his willingness to volunteer and hope he can join us for a future event. To ensure the food drive's success, I will personally reach out to our volunteer network to find a replacement.

7. Non-Functional Requirements

7.1. Security Aspects

- The system shall store API keys securely using environment variables (.env file) and avoid hardcoding sensitive information.
- The system shall not store any personally identifiable user data, ensuring basic user privacy.
- The backend shall follow a stateless architecture to minimize session-related vulnerabilities.
- The system shall restrict direct access to internal data sources such as the vector database.
- Advanced security features like authentication, authorization, and encryption are not implemented and can be considered as future enhancements.

7.2. Performance Aspects

- Low latency response generation using Groq's high-performance LLM inference.
- Efficient multi-agent orchestration enabling parallel and structured task execution.
- Fast entity extraction using spaCy for lightweight and real-time NER processing.
- Scalable API architecture using FastAPI to handle concurrent requests efficiently.
- Optimized request handling through asynchronous processing and minimal blocking operations.
- Reduced manual workload by automating classification, extraction, and response generation.
- High throughput capability allows the system to process multiple communications simultaneously.
- Robust error handling to prevent system crashes and ensure continuous availability.
- Modular architecture enables easy scaling and integration with additional components.
- Improved user experience with responsive frontend and quick feedback loops.

8. References

[1] Groq Inc., "Groq API Documentation." [Online]. Available: https://console.groq.com/docs. [Accessed: Mar. 2026].

[2] CrewAI, "CrewAI Documentation." [Online]. Available: https://docs.crewai.com/. [Accessed: Mar. 2026].

[3] FastAPI, "FastAPI Documentation." [Online]. Available: https://fastapi.tiangolo.com/. [Accessed: Mar. 2026].

[4] spaCy, "spaCy Usage and Documentation." [Online]. Available: https://spacy.io/usage. [Accessed: Mar. 2026].

[5] React Team, "React Documentation." [Online]. Available: https://react.dev/. [Accessed: Mar. 2026].

[6] Tailwind Labs, "Tailwind CSS Documentation." [Online]. Available: https://tailwindcss.com/docs. [Accessed: Mar. 2026].

[7] Axios, "Axios HTTP Client Documentation." [Online]. Available: https://axios-http.com/docs/intro. [Accessed: Mar. 2026].

[8] Uvicorn, "Uvicorn ASGI Server Documentation." [Online]. Available: https://www.uvicorn.org/. [Accessed: Mar. 2026].

[9] Python Software Foundation, "Python Documentation." [Online]. Available: https://docs.python.org/3/. [Accessed: Mar. 2026].